# Laboratorium 5 -Three.js: dynamiczna plansza i kamera podążająca za graczem

## 1. Wprowadzenie teoretyczne

### 1.1. Dynamiczna plansza

W dotychczasowej wersji Road Hoppera wszystkie pasy trawy i drogi były zdefiniowane przed uruchomieniem gry w tablicy rows. Położenia samochodów i drzew pochodziły z tablic scene.cars oraz scene.trees, a zakończenie rozgrywki następowało po dotarciu do ostatniego pasa. Jest to przykład **planszy statycznej** - jej wielkość i zawartość są z góry określone.

**Generowanie proceduralne** oznacza tworzenie elementów świata na podstawie reguł programu. W tym laboratorium każdemu nowemu pasowi nadamy pozycję na osi Z, wybierzemy jego rodzaj, a następnie opcjonalnie dodamy samochód albo drzewo. Powstawanie pasów będzie zależało od aktualnej pozycji postaci. Nie generujemy całej nieskończonej mapy: przechowujemy jedynie niewielki obszar znajdujący się w pobliżu gracza.

Ważnym elementem będzie **zarządzanie cyklem życia obiektu**. Dodanie pasa wymaga zarówno utworzenia danych w rows, jak i wyświetlenia jego reprezentacji w scenie. Usunięcie pasa wymaga wykonania obu operacji w przeciwnym kierunku. Podobna zasada dotyczy samochodów oraz drzew. Pozwala to utrzymać względnie stałą liczbę aktywnych obiektów, mimo że rozgrywka może trwać coraz dłużej.

### 1.2. Kamera śledząca

Drugim zagadnieniem jest **kamera śledząca**. W Laboratorium 4 ustawienie kamery było stałe. Teraz pozycja kamery i punkt, na który patrzy, będą stopniowo przesuwane wraz z graczem. Wykorzystamy interpolację (lerp), aby uniknąć gwałtownych zmian widoku. Wciąż będziemy korzystać z projekcji ortograficznej znanej z wcześniejszych zajęć.

**Powiązanie z WebGL2.** Macierz View nadal określa położenie obserwatora, a macierz Projection sposób rzutowania sceny. Three.js oblicza je za nas na podstawie obiektu Camera. Podobnie dotychczasowe transformacje i dane o położeniu pozostają aktualne; zmienia się sposób zarządzania obiektami na scenie.

## 2. Przygotowanie projektu do trybu dynamicznie generowanej planszy

**2.1.** Otwórz własny projekt w Visual Studio Code, a następnie uruchom dotychczasowy serwer poleceniem

```
npm run dev.
```

**2.2.** W src/main.js dodaj tymczasową stałą decydującą o wersji rozgrywki. Na kolejnym laboratorium zastąpimy ją wyborem z menu:

```
const GAME_MODE = 'endless';
```

W trybie endless kolejne pasy będą generowane wraz z postępem gracza, a rozgrywka nie będzie kończyć się po dotarciu do ostatniego pasa początkowej planszy. Punktacja będzie uzależniona od pokonanego dystansu.

## 3. Kamera podążająca za postacią

W poprzednim laboratorium camera3D.position i camera3D.lookAt() otrzymały stałe współrzędne. Teraz chcemy, aby kamera utrzymywała podobną odległość od gracza podczas przekraczania kolejnych pasów. Zamiast natychmiast ustawiać nową pozycję, obliczymy ją stopniowo w każdej klatce animacji.

**1.**Zastosuj poniższe zmiany w pliku **main.js**

**1.1.** Przygotuj wektor przechowujący jej aktualny punkt obserwacji:

```
const cameraTarget = new THREE.Vector3(0, -0.3, scene.player.position[2]-1.05);
```

**1.2.** Dopisz poniżej funkcję updateFollowCamera(dt). *Parametr dt jest czasem, który upłynął od poprzedniej klatki - został już wprowadzony na zajęciach z WebGL2:*

```
function updateFollowCamera(dt) {
   const p = scene.player.position;
   const alpha = 1 - Math.exp(-5 * dt);
   const wantedPosition = new THREE.Vector3(p[0] * 0.25 + 2.0, 3, p[2] + 2.5);
   const wantedTarget = new THREE.Vector3(p[0] * 0.25, -0.3, p[2] - 1.05);

   camera3D.position.lerp(wantedPosition, alpha);
   cameraTarget.lerp(wantedTarget, alpha);
   camera3D.lookAt(cameraTarget);
}
```

Współczynnik alpha określa, jak szybko kamera zbliża się do docelowej pozycji. Zastosowanie **1 - Math.exp(-5 * dt)** daje podobne tempo przejścia przy różnych liczbach klatek na sekundę. Wartość p[0] * 0.25 oznacza, że kamera reaguje tylko częściowo na przesunięcia gracza w lewo i w prawo.

**1.3.** W istniejącej funkcji frame(time) wywołaj tę funkcję **przed** render(), ale tylko dla trybu endless, aktywnej gry i wyłączonej pauzy

```
if (GAME_MODE === 'endless' &&
   !isPaused && gameState.status === 'playing') {
   updateFollowCamera(dt);
}
```

**Eksperyment 1.**

Sprawdź, jak zmiana parametrów funkcji updateFollowCamera() wpływa na zachowanie kamery podczas ruchu gracza.

- Zmień wartość 5 we wzorze 1 - Math.exp(-5 * dt) kolejno na 2 i 10. Za każdym razem przejdź kilka pasów i zaobserwuj, jak szybko kamera podąża za graczem.
- Zmień odsunięcie kamery od gracza z p[2] + 2.5 na p[2] + 3.2. Sprawdź, jak zmienia się widok planszy.
- Podczas ruchu kamery naciśnij P i sprawdź, czy pauza zatrzymuje jej przemieszczanie.

Po zakończeniu eksperymentu przywróć początkowe wartości 5 oraz 2.5.

## 4. Dane potrzebne do generowania kolejnych pasów

W Lab 3 numer odwiedzonego pasa był wyznaczany na podstawie jego pozycji w tablicy. Przy usuwaniu starszych elementów taki indeks przestaje być stały: po wykonaniu splice() pozostałe pasy przesuwają się w tablicy. Dlatego każdy pas otrzyma **niezmienny numer laneIndex**, który odzwierciedla jego kolejność w świecie, a nie pozycję w aktualnej tablicy.




Schemat poglądowy: liczba aktywnych pasów jest ograniczona parametrami AHEAD i BEHIND.

**4.1.** Utwórz plik **src/three/world.js** i zastosuj poniższe zmiany w pliku

**4.1.1.** W poprzednim laboratorium szerokość pasów oraz ich położenie były określane bezpośrednio w kodzie. Przed rozpoczęciem generowania dynamicznej planszy ujednolicimy te wartości, przyjmując szerokość każdego pasa w osi Z równą 0.35 jednostki. W pliku objects.js sprawdź, czy geometria pasa ma następujące wymiary:

```
const rowGeometry = new THREE.BoxGeometry(1.8, 0.04, 0.35);
```

W pliku sceneData.js upewnij się również, że kolejne pasy są rozmieszczone co 0.35 jednostki w osi Z, np. 0.70, 0.35, 0.00, -0.35. W razie potrzeby dostosuj ich pozycje oraz pozycje znajdujących się na nich obiektów.

Następnie w nowym pliku **world.js** zaimportuj istniejące dane gry i zadeklaruj parametry generatora:

```
import { rows, scene as gameScene } from '../game/sceneData.js';

const STEP = 0.35;
const AHEAD = 5;
const BEHIND = 3;
const INITIAL_ROWS = rows.length;
const FIRST_Z = Math.max(...rows.map(row => row.z));
let nextIndex = INITIAL_ROWS;
```

Kluczowe informacje:

- Wartość STEP - określa odległość między środkami kolejnych pasów. Wartość 0.35 odpowiada ich szerokości, dzięki czemu pasy będą przylegały do siebie bez przerw.
- AHEAD - określa liczbę pasów przygotowywanych przed graczem,
- BEHIND - liczbę pasów, które należy zachować za postacią. Początkowych pasów z Lab 4 nie usuwamy: będą służyły do bezpiecznego rozpoczęcia i restartu gry.
- INITIAL_ROWS - zapamiętuje liczbę pasów początkowej planszy.

- FIRST_Z - przechowuje pozycję Z pierwszego pasa, będącą punktem odniesienia dla generatora.
- nextIndex - przechowuje numer kolejnego pasa, który zostanie wygenerowany.

**4.1.2.** Utwórz funkcję numerującą istniejące pasy. Ma ona zostać wywołana tylko raz, przed utworzeniem planszy w Three.js:

```
export function prepareWorld() {
   rows.sort((a, b) => b.z - a.z);
   rows.forEach((row, index) => {
   row.laneIndex = index;
   });
}
```

Funkcja najpierw sortuje istniejące pasy według współrzędnej Z, a następnie przypisuje im kolejne numery, zaczynając od 0. Dzięki temu numeracja będzie rosła zgodnie z kierunkiem przemieszczania się gracza do przodu, czyli w stronę ujemnej osi Z. Funkcję prepareWorld() wywołamy tylko raz, przed utworzeniem początkowej planszy w Three.js.

**4.2.** W istniejącym main.js zaimportuj funkcje prepareWorld() i wywołaj je **przed** createBasicObjects(scene3D). Dzięki temu także początkowe pasy będą miały własne numery. Nie przepisuj ręcznie obiektów rows, ponieważ każdy student mógł dodać inne pasy podczas WebGL2.

**Eksperyment 2.**

Wypisz w konsoli pary row.laneIndex i row.z, a następnie sprawdź, czy numery rosną w kierunku ruchu postaci do przodu (ujemna oś Z). Wyjaśnij, dlaczego samo rows.indexOf(row) nie byłoby wystarczające po usunięciu fragmentu świata.

# 5. Dynamiczne tworzenie geometrii terenu

W poprzednim laboratorium wszystkie pasy drogi i trawy były tworzone jednorazowo podczas uruchamiania gry. Aby móc generować kolejne fragmenty planszy podczas rozgrywki, przygotujemy funkcję addRowVisual(), która będzie odpowiadała za tworzenie pojedynczego pasa.Wykorzystamy wspólną geometrię i materiały, dzięki czemu nie trzeba będzie przygotowywać ich ponownie dla każdego nowego fragmentu terenu.

**5.1.** Otwórz **src/three/objects.js**. W obiekcie objects dodaj właściwość rows: new Map(), która będzie przechowywać odwołania do utworzonych pasów:

```
export const objects = {
   playerRoot: null,
   cars: [],
   trees: [],
   rows: new Map(),
};
```

**5.2.** W tym samym pliku, nad funkcją createBasicObjects(scene3D), przygotuj wspólną geometrię oraz materiały dla pasów drogi, trawy i oznaczeń jezdni:

```
const rowGeometry = new THREE.BoxGeometry(1.8, 0.04, 0.35);
const rowMaterials = {
  road: new THREE.MeshBasicMaterial({ color: 0x30323a }),
  grass: new THREE.MeshBasicMaterial({ color: 0x2ba344 }),
};

const lineGeometry = new THREE.BoxGeometry(0.15, 0.006, 0.018);
const lineMaterial = new THREE.MeshBasicMaterial({ color: 0xffffff });
```

Następnie poniżej dodaj funkcję addRowVisual(), która będzie odpowiedzialna za utworzenie pojedynczego pasa:

```
export function addRowVisual(scene3D, row) {
  const root = new THREE.Group();

  const mesh = new THREE.Mesh(rowGeometry, rowMaterials[row.type]);
  mesh.position.set(0, GROUND_Y - 0.02, row.z);
  root.add(mesh);

  const previous = rows.find(r => r.laneIndex === row.laneIndex - 1);

  if (row.type === 'road' && previous?.type === 'road' &&
    Math.abs(previous.z - row.z - 0.35) < 0.001) {

    const lineZ = (previous.z + row.z) / 2;

    for (let x = -0.75; x <= 0.75; x += 0.30) {
      const line = new THREE.Mesh(lineGeometry, lineMaterial);
      line.position.set(x, GROUND_Y + 0.003, lineZ);
      root.add(line);
    }
  }

  scene3D.add(root);
  objects.rows.set(row.laneIndex, root);
}
```

Funkcja tworzy pas z odpowiednim materiałem, a następnie sprawdza, czy poprzedni pas również jest drogą. Jeżeli dwa pasy drogi bezpośrednio ze sobą sąsiadują, na ich granicy powstaje przerywana biała linia.
Zastosowanie THREE.Group pozwala połączyć teren i jego oznaczenia w jeden obiekt. Dzięki temu podczas późniejszego usuwania pasa z planszy usuniemy również przypisane do niego linie.

**5.3.** W pliku objects.js odszukaj funkcję createBasicObjects(scene3D). Usuń z niej dotychczasową deklarację geometrii pasów (rowGeometry lub grassGeometry i roadGeometry, zależnie od wersji projektu) oraz pętlę for, w której ręcznie tworzony jest obiekt rowMesh. W miejsce usuniętego fragmentu wstaw:

```
for (const row of rows) {
  addRowVisual(scene3D, row);
}
```

Usuń również dotychczasowy fragment odpowiedzialny za ręczne tworzenie przerywanej linii między początkowymi pasami drogi. Nie jest już potrzebny, ponieważ oznaczenia tworzy funkcja addRowVisual(). Pozostały kod funkcji, w tym tworzenie postaci gracza, pozostaw bez zmian. Po uruchomieniu gry początkowa plansza powinna wyglądać tak samo jak wcześniej, ale jej pasy i oznaczenia będą już tworzone przez nową funkcję.

**5.4.** W pliku **world.js** dopisz import funkcji addRowVisual z pliku objects.js.

```
import { addRowVisual } from './objects.js';
```

Pod funkcją prepareWorld(), dodaj funkcję wyszukującą aktualny pas gracza i funkcję przygotowującą kolejne pasy gry:

```
function playerLane() {
   const row = rows.find(r =>
   Math.abs(r.z - gameScene.player.position[2]) < 0.001);
   return row?.laneIndex ?? 0;
}

export function extendWorld(scene3D, carTemplate, treeTemplate) {
   const current = playerLane();

   while (nextIndex <= current + AHEAD) {
      const index = nextIndex++;
      let type = Math.random() < 0.45 ? 'road' : 'grass';

      const lastType = rows[rows.length - 1].type;
      let streak = 0;

      for (let i = rows.length - 1; i >= 0; i--) {
         if (rows[i].type !== lastType) break;
         streak++;
      }

      if (lastType === 'road' && streak >= 5) type = 'grass';
      if (lastType === 'grass' && streak >= 3) type = 'road';
      const row = { type, z: FIRST_Z - index * STEP,
      laneIndex: index };
      rows.push(row);
      addRowVisual(scene3D, row);
   }
```

```
}
```

Funkcja playerLane() wyszukuje pas, na którym aktualnie znajduje się gracz, i zwraca jego numer laneIndex. Jeśli pas nie zostanie odnaleziony, zwraca 0.

Funkcja extendWorld() wykorzystuje ten numer do sprawdzenia, czy przed graczem znajduje się wystarczająca liczba pasów. Jeśli nie, generuje kolejne, losując ich rodzaj i obliczając pozycję na osi Z. Każdy nowy pas jest dodawany zarówno do tablicy rows, jak i do sceny Three.js.

Wartość 0.45 oznacza 45% początkowego prawdopodobieństwa wylosowania pasa drogi. Aby uniknąć zbyt długich fragmentów tego samego rodzaju terenu, wprowadzono dodatkowe ograniczenia. Zmienna lastType przechowuje rodzaj ostatniego pasa, natomiast streak zlicza liczbę kolejnych pasów tego samego rodzaju. Po osiągnięciu limitu 5 pasów drogi lub 3 pasów trawy generator wymusza zmianę rodzaju terenu.

Na tym etapie generujemy jedynie teren. Parametry carTemplate i treeTemplate wykorzystamy w punkcie 6 do tworzenia samochodów i drzew.

**5.5.** Podłączenie generatora do głównej pętli gry

Na początku pliku znajdź istniejący import:

```
import { prepareWorld } from './three/world.js';
```

Zamień na:

```
import { prepareWorld, extendWorld } from './three/world.js';
```

W tym samym pliku znajdź wywołanie:

```
addModels(scene3D, carTemplate, treeTemplate);
```

Bezpośrednio pod nim dodaj:

```
if (GAME_MODE === 'endless') {
  extendWorld(scene3D, carTemplate, treeTemplate);
}
```

Dzięki temu przed uruchomieniem pierwszej klatki zostanie przygotowany początkowy fragment generowanego świata.

Teraz w istniejącej funkcji frame(time) znajdź warunek odpowiedzialny za aktualizację kamery w trybie endless i dopisz w nim kolejne wywołanie funkcji extendWorld pod wywołaniem funkcji updateFollowCamera.

**5.6.** W dotychczasowej wersji gry dotarcie do ostatniego pasa powodowało zakończenie rozgrywki. Ponieważ przygotowujemy tryb endless, usuniemy ten warunek, aby gracz mógł swobodnie przemieszczać się po kolejnych generowanych pasach.

Otwórz plik src/game/gameRules.js. W funkcji updateGameState() znajdź i usuń poniższy fragment:

```
if (lane === lanes.length - 1) {
  gameState.status = 'won';
```

```
}
```

Pozostały kod funkcji pozostaw bez zmian. Dotarcie do ostatniego pasa początkowej planszy nie będzie już zatrzymywać rozgrywki. Punktację dostosujemy do dynamicznie generowanej planszy w punkcie 8.

Eksperyment 3. Wpływ parametru AHEAD na generowanie planszy.
Sprawdź, jak wartość AHEAD wpływa na liczbę pasów przygotowywanych przed graczem.

- Zmień AHEAD z 5 na 2. Uruchom ponownie grę, przejdź kilka pasów i zaobserwuj, kiedy pojawiają się nowe fragmenty planszy.
- Następnie ustaw AHEAD = 8 i powtórz eksperyment. Sprawdź, jak zmieniła się liczba pasów widocznych przed graczem.
- Porównaj oba ustawienia i zastanów się, jakie zalety i wady ma generowanie większej liczby pasów z wyprzedzeniem.

Po zakończeniu eksperymentu przywróć wartość AHEAD = 5.

## 6. Dynamiczne generowanie samochodów i drzew

Dotychczas funkcja addModels() tworzyła wszystkie samochody i drzewa na początku rozgrywki. Aby móc dodawać przeszkody również na nowych pasach, przygotujemy funkcje tworzące pojedyncze obiekty, a następnie wykorzystamy je w generatorze świata. Nadal będziemy korzystać z wcześniej wczytanych modeli GLB.

**6.1.** W pliku **objects.js** nad funkcją addModels(), dodaj dwie funkcje:

```
export function addCarVisual(scene3D, car, carTemplate) {
   const root = makeInstance(carTemplate, 0.40);
   root.rotation.y = car.speed < 0 ? Math.PI / 2 : -Math.PI / 2;
   scene3D.add(root);
   objects.cars.push(root);
}

export function addTreeVisual(scene3D, tree, treeTemplate) {
   const root = makeInstance(treeTemplate, 0.70);
   scene3D.add(root);
   objects.trees.push(root);
}
```

Funkcje te tworzą pojedyncze obiekty, dodają je do sceny i zapisują w odpowiednich tablicach. Dzięki temu możemy je wywoływać wielokrotnie, bez ponownego ładowania plików GLB.

**6.2.** W istniejącej funkcji addModels() zachowaj obie pętle for, ale usuń z ich wnętrza kod odpowiedzialny za tworzenie modeli. Zamiast niego wykorzystaj przygotowane funkcje:

- W pętli samochodów: `addCarVisual(scene3D, car, carTemplate);`
- W pętli drzew: `addTreeVisual(scene3D, tree, treeTemplate);`

Funkcja addModels() nadal utworzy początkowe obiekty, natomiast nowe funkcje wykorzystamy również do generowania przeszkód na kolejnych pasach.

**6.3.** W pliku world.js rozbudujemy funkcję extendWorld() o generowanie samochodów i drzew. Wykorzystamy przygotowane wcześniej funkcje addCarVisual() oraz addTreeVisual(), dzięki

czemu nie będziemy musieli ponownie ładować modeli GLB. Najpierw uzupełnij istniejący import z pliku objects.js oraz dodaj import stanu gry:

```
import { addRowVisual, addCarVisual, addTreeVisual} from './objects.js';
import {gameState} from '../game/gameRules.js';
```

W funkcji extendWorld(), wewnątrz pętli while, bezpośrednio po instrukcji addRowVisual(scene3D, row);, dodaj fragment odpowiedzialny za generowanie samochodów:

```
if (type === 'road') {
   const direction = Math.random() < 0.5 ? -1 : 1;
   const baseSpeed = direction * (0.28 + Math.random() * 0.20);

   const car = {
      position: [Math.random() * 1.6 - 0.8, 0, row.z],
      speed: baseSpeed * gameState.speedMultiplier,
      baseSpeed,
      laneIndex: index
   };

   gameScene.cars.push(car);
   addCarVisual(scene3D, car, carTemplate);
}
```

Na każdym nowym pasie drogi tworzony jest jeden samochód. Jego pozycja na osi X, kierunek jazdy oraz prędkość są losowane. Zmienna baseSpeed przechowuje początkową prędkość pojazdu, natomiast gameState.speedMultiplier pozwala dostosować ją do aktualnego poziomu trudności. Właściwość laneIndex określa pas, do którego przypisany jest samochód.

Następnie bezpośrednio po powyższym bloku if dodaj generowanie drzew:

```
else {
   const tree = {
      position: [Math.random() < 0.5 ? -0.65 : 0.65, 0,row.z],
      laneIndex: index
   };
   gameScene.trees.push(tree);
   addTreeVisual(scene3D, tree, treeTemplate);
}
```

Na każdym wygenerowanym pasie trawy pojawia się jedno drzewo, umieszczone losowo przy lewej lub prawej krawędzi. Dzięki temu środkowa część pasa pozostaje wolna dla gracza. W zadaniach do sprawozdania generator zostanie rozbudowany o możliwość umieszczania większej liczby drzew oraz dodatkowe zasady ich rozmieszczania.

6.4 W poprzedniej wersji gry funkcja updateDifficulty() korzystała z tablicy startSpeeds, zawierającej początkowe prędkości samochodów. Ponieważ teraz pojazdy są również tworzone podczas rozgrywki, musimy uwzględnić ich prędkości przy aktualizacji poziomu trudności.

Nowe samochody przechowują swoją początkową prędkość we właściwości baseSpeed, którą dodaliśmy w poprzednim punkcie.

Otwórz plik src/game/gameRules.js. W funkcji updateDifficulty() odszukaj dotychczasową pętlę aktualizującą prędkości samochodów:

```
scene.cars.forEach((car, index) => {
   car.speed = startSpeeds[index] * factor;
});
```

Zastąp ją następującym fragmentem:

```
scene.cars.forEach((car, index) => {
   const baseSpeed = index < startSpeeds.length
      ? startSpeeds[index]
      : car.baseSpeed;

   car.speed = baseSpeed * factor;
});
```

Warunek sprawdza, czy samochód należał do początkowej planszy. W takim przypadku wykorzystuje prędkość zapisaną w tablicy startSpeeds. Dla samochodów wygenerowanych dynamicznie pobierana jest natomiast wartość car.baseSpeed.

Dzięki temu wzrost poziomu trudności wpływa na wszystkie samochody, niezależnie od momentu ich utworzenia. Zachowany zostaje również pierwotny kierunek jazdy każdego pojazdu.

**Eksperyment 4.** Zmień prawdopodobieństwo wygenerowania drogi z 0.45 na 0.70, a następnie zmodyfikuj zakres losowanych prędkości samochodów. Obserwuj, jak zmieniają się zagęszczenie ruchu i poziom trudności rozgrywki. Po zakończeniu eksperymentu przywróć wartości początkowe.

## 7. Usuwanie niepotrzebnych fragmentów planszy

Dotychczas przygotowany generator tworzy nowe pasy drogi i trawy wraz z samochodami oraz drzewami, gdy gracz przemieszcza się do przodu. Każdy nowy element jest dodawany do sceny Three.js oraz odpowiednich tablic przechowujących dane gry. Jednak bez usuwania starszych fragmentów planszy liczba obiektów będzie stale rosła, nawet jeśli gracz nie będzie już ich widział. W dłuższej rozgrywce może to prowadzić do zwiększonego zużycia pamięci oraz niepotrzebnego przetwarzania obiektów znajdujących się daleko za postacią. Aby temu zapobiec, wprowadzimy mechanizm usuwania fragmentów świata, które nie są już potrzebne.

Usuwanie musi obejmować zarówno reprezentację graficzną obiektu w Three.js, jak i odpowiadające mu dane gry. W przeciwnym razie niewidoczne samochody mogłyby nadal uczestniczyć w obliczeniach ruchu i wykrywaniu kolizji.

7.1. Otwórz plik src/three/world.js i znajdź istniejący import:

```
import { addRowVisual, addCarVisual, addTreeVisual } from './objects.js';
```

Uzupełnij go o obiekt objects, który przechowuje odwołania do modeli znajdujących się na scenie:

```
import { addRowVisual, addCarVisual, addTreeVisual, objects } from './objects.js';
```

Pod istniejącą funkcją extendWorld() rozpocznij tworzenie nowej funkcji removeGeneratedRow().

```
function removeGeneratedRow(scene3D, row) {
   for (let i = gameScene.cars.length - 1; i >= 0; i--) {
      if (gameScene.cars[i].laneIndex !== row.laneIndex) continue;

      scene3D.remove(objects.cars[i]);
      objects.cars.splice(i, 1);
      gameScene.cars.splice(i, 1);
   }
}
```

Pętla wyszukuje samochody należące do wskazanego pasa i usuwa je ze sceny oraz obu tablic. Iterujemy od końca, ponieważ splice() zmienia indeksy elementów.

Dla znalezionych samochodów wykonywane są trzy operacje:

- scene3D.remove()- usuwa model samochodu ze sceny Three.js.
- objects.cars.splice()- usuwa odwołanie do modelu z tablicy obiektów graficznych.
- gameScene.cars.splice()- usuwa dane samochodu z logiki gry.

W tej samej funkcji, bezpośrednio po pętli usuwającej samochody, dodaj analogiczny mechanizm dla drzew:

```
for (let i = gameScene.trees.length - 1; i >= 0; i--) {
   if (gameScene.trees[i].laneIndex !== row.laneIndex) continue;

   scene3D.remove(objects.trees[i]);
   objects.trees.splice(i, 1);
   gameScene.trees.splice(i, 1);
}
```

Po usunięciu samochodów i drzew pozostaje usunąć sam pas. Na końcu funkcji removeGeneratedRow(), przed jej zamykającym nawiasem klamrowym, dodaj:

```
scene3D.remove(objects.rows.get(row.laneIndex));
objects.rows.delete(row.laneIndex);
rows.splice(rows.indexOf(row), 1);
```

Instrukcja objects.rows.get() pobiera grupę THREE.Group odpowiadającą wskazanemu pasowi. Usunięcie tej grupy ze sceny powoduje również zniknięcie przypisanych do niej oznaczeń drogowych.

Następnie metoda delete() usuwa odwołanie z kolekcji Map, a splice() usuwa informacje o pasie z tablicy rows.

**Ważne:** Nie wywołujemy tutaj geometry.dispose() ani material.dispose(), ponieważ wiele obiektów korzysta ze współdzielonych geometrii i materiałów. Zwolnienie takich zasobów mogłoby wpłynąć na pozostałe modele znajdujące się na scenie.

7.2 Wykorzystamy funkcję removeGeneratedRow(), aby automatycznie usuwać pasy znajdujące się zbyt daleko za graczem. Zachowamy trzy poprzednie pasy, zgodnie z wartością BEHIND = 3. Początkowych pasów nie będziemy usuwać, aby umożliwić ponowne rozpoczęcie gry.

W pliku world.js, na końcu funkcji extendWorld(), bezpośrednio po zakończeniu pętli while, dodaj:

```
for (const row of [...rows]) {
  if (row.laneIndex < INITIAL_ROWS) continue;

  if (row.laneIndex < current - BEHIND) {
    removeGeneratedRow(scene3D, row);
  }
}
```

Pętla przegląda istniejące pasy i usuwa te, które znajdują się dalej niż BEHIND pasów za graczem. Warunek z INITIAL_ROWS zabezpiecza początkową planszę przed usunięciem.

Zapis [...rows] tworzy kopię tablicy, dzięki czemu możemy bezpiecznie usuwać elementy z oryginalnej tablicy podczas iteracji.

7.3 Po usunięciu starszych pasów między początkową planszą a aktualnym obszarem gry może powstać przerwa. Dotychczasowa funkcja movePlayer() mogłaby pozwolić graczowi przeskoczyć przez nią. Aby temu zapobiec, sprawdzimy odległość między sąsiednimi pasami.

W pliku src/game/gameLogic.js dodaj stałą określającą rozstaw pasów:

```
const STEP = 0.35;
```

W funkcji movePlayer() znajdź warunki obsługujące ArrowUp i ArrowDown. Zastąp je poniższym fragmentem:

```
if (key === 'ArrowUp' && index >= 0 && index < lanes.length - 1) {
  const nextZ = lanes[index + 1];

  if (Math.abs(nextZ - p[2]) <= STEP + 0.001) {
    p[2] = nextZ;
  }
}

if (key === 'ArrowDown' && index > 0) {
  const nextZ = lanes[index - 1];

  if (Math.abs(nextZ - p[2]) <= STEP + 0.001) {
    p[2] = nextZ;
  }
}
```

Zmienna nextZ przechowuje pozycję sąsiedniego pasa. Ruch zostanie wykonany tylko wtedy, gdy odległość między pasami nie przekracza 0.35 jednostki (z niewielką tolerancją). Dzięki temu gracz nie może przeskakiwać przez usunięte fragmenty planszy.

# 8. Punktacja i restart gry

W dotychczasowej wersji gry zasady punktacji i restartu były dostosowane do statycznej planszy. Teraz zmodyfikujemy je tak, aby uwzględniały dynamicznie generowane pasy oraz samochody, zachowując dotychczasowy mechanizm utraty życia i zwiększania trudności.

8.1 W pliku src/game/gameRules.js funkcja updateGameState() korzystała dotychczas z tablicy lanes, utworzonej tylko raz podczas uruchamiania gry. Nie uwzględnia więc nowo wygenerowanych pasów.

Odszukaj w tej funkcji fragment:

```
const lane = lanes.findIndex(z =>
   Math.abs(z - scene.player.position[2]) < 0.001
);
```

A następnie zastąp go kodem:

```
const row = rows.find(r =>
   Math.abs(r.z - scene.player.position[2]) < 0.001
);

const lane = row?.laneIndex ?? -1;
```

Zamiast korzystać z początkowej tablicy lanes, wyszukujemy aktualny pas bezpośrednio w rows. Jego właściwość laneIndex pozwala prawidłowo naliczać punkty również za dynamicznie generowane pasy. Pozostały kod funkcji updateGameState(), w tym naliczanie punktów, utrata życia i zwiększanie trudności, pozostaw bez zmian.

8.2 Dotychczasowa funkcja restartGame() przywracała pozycje wszystkich samochodów. Ponieważ teraz tworzymy pojazdy również podczas rozgrywki, musimy ograniczyć tę operację do samochodów istniejących na początku gry.

W pliku src/game/gameRules.js, wewnątrz funkcji restartGame(), znajdź fragment:

```
scene.cars.forEach((car, index) => {
   car.position.splice(0, 3, ...startCars[index]);
   car.speed = startSpeeds[index];
});
```

Zastąp go następującym kodem:

```
scene.cars.slice(0, startCars.length).forEach((car, index) => {
   car.position.splice(0, 3, ...startCars[index]);
   car.speed = startSpeeds[index];
});
```

Metoda slice() ogranicza aktualizację do początkowych samochodów, dla których zapisaliśmy oryginalne pozycje i prędkości. Nowe samochody zostaną usunięte wraz z wygenerowanymi pasami w kolejnym punkcie.

8.3 Podczas restartu gry musimy usunąć wszystkie dynamicznie wygenerowane pasy wraz z samochodami i drzewami, a następnie odtworzyć początkowy fragment planszy.

```
export function resetWorld(scene3D, carTemplate, treeTemplate) {
   for (const row of [...rows]) {
      if (row.laneIndex >= INITIAL_ROWS) {
         removeGeneratedRow(scene3D, row);
      }
   }

   nextIndex = INITIAL_ROWS;
   extendWorld(scene3D, carTemplate, treeTemplate);
}
```

Funkcja usuwa wygenerowane pasy za pomocą przygotowanej wcześniej removeGeneratedRow(). Początkowe pasy pozostają bez zmian. Następnie przywracamy licznik nextIndex i ponownie wywołujemy generator, aby przygotować nowe fragmenty świata przed graczem.

8.4 W main.js mamy już obsługę restartu. Musimy ją jedynie rozszerzyć o wywołanie funkcji resetWorld() oraz przywrócenie kamery do początkowej pozycji.

W pliku src/main.js uzupełnij import z world.js o funkcję resetWorld():

```
import { prepareWorld, extendWorld, resetWorld } from './three/world.js';
```

W istniejącej obsłudze zdarzenia keydown znajdź warunek odpowiedzialny za klawisz R i zastąp go poniższym fragmentem:

```
if (event.code === 'KeyR') {
   if (event.repeat) return;
   restartGame();

   if (GAME_MODE === 'endless') {
      resetWorld(scene3D, carTemplate, treeTemplate);
      const p = scene.player.position;
      camera3D.position.set(
         p[0] * 0.25 + 2.0, 3, p[2] + 2.5
      );

      cameraTarget.set(
         p[0] * 0.25, -0.3, p[2] - 1.05
      );

      camera3D.lookAt(cameraTarget);
   }
```

```
    isPaused = false;
    previousTime = null;
    return;
}
```

Funkcja restartGame() przywraca stan rozgrywki, natomiast resetWorld() usuwa wygenerowane pasy i tworzy nowy fragment planszy. Następnie kamera wraca do pozycji początkowej, a pauza zostaje wyłączona.

Wyzerowanie previousTime zapobiega nagłemu przeskokowi samochodów po wznowieniu rozgrywki.

8.5 Po utracie życia gracz wraca na początkowy pas. Musimy również przywrócić kamerę i wygenerowane fragmenty świata, aby kontynuowanie rozgrywki było możliwe. W pliku main.js, pod funkcją updateFollowCamera(), dodaj funkcję przywracającą kamerę do pozycji gracza:

```
function resetFollowCamera() {
    const p = scene.player.position;

    camera3D.position.set(p[0] * 0.25 + 2.0, 3, p[2] + 2.5);
    cameraTarget.set(p[0] * 0.25, -0.3, p[2] - 1.05);
    camera3D.lookAt(cameraTarget);
}
```

W istniejącej obsłudze klawisza R zastąp fragment ustawiający camera3D.position, cameraTarget i camera3D.lookAt() pojedynczym wywołaniem:

```
resetFollowCamera();
```

W funkcji frame(time) znajdź fragment:

```
if (!isPaused && gameState.status === 'playing') {
    updateCars(dt);
    updateGameState();
}
```

Zastąp go następującym:

```
if (!isPaused && gameState.status === 'playing') {
    const livesBefore = gameState.lives;
    updateCars(dt);
    updateGameState();
    if (GAME_MODE === 'endless' &&
        gameState.lives < livesBefore &&
        gameState.status === 'playing') {

        resetWorld(scene3D, carTemplate, treeTemplate);
        resetFollowCamera();
    }
}
```

Zmienna livesBefore przechowuje liczbę żyć przed aktualizacją stanu gry. Jeżeli liczba żyć spadnie, a rozgrywka nadal trwa, przywracamy wygenerowany świat i kamerę. Po utracie ostatniego życia gra pozostaje zakończona i oczekuje na restart klawiszem R.

# 9. Zadania do sprawozdania

Wykonaj samodzielnie poniższe zadania, rozbudowując mechanizmy dynamicznego generowania planszy przygotowane podczas laboratorium.

**Zadanie 1. Generowanie drzew i wykrywanie kolizji.**

Zmodyfikuj generator terenu w taki sposób, aby na pasach trawy mogła pojawiać się losowa liczba drzew (od 1 do 3). Drzewa powinny być rozmieszczane w różnych miejscach pasa, również w jego środkowej części. Zadbaj o to, aby ich rozmieszczenie nie blokowało całkowicie możliwości przejścia na kolejny pas.

Dodatkowo zaimplementuj mechanizm kolizji z drzewami. Gracz nie powinien mieć możliwości wejścia na pozycję zajętą przez drzewo. Próba wykonania takiego ruchu powinna zostać zablokowana, bez utraty życia.

**Zadanie 2. Zróżnicowany ruch samochodów.**

Rozbuduj generator samochodów tak, aby na wybranych pasach drogi mogło pojawiać się od 1 do 3 pojazdów. Samochody powinny otrzymywać losowe pozycje początkowe oraz zróżnicowane prędkości, zachowując wspólny kierunek jazdy w obrębie danego pasa.

Dostosuj istniejącą logikę ruchu i rozmieszczania pojazdów tak, aby samochody utrzymywały odpowiednie odstępy, nie nakładały się na siebie i prawidłowo pojawiały się ponownie po opuszczeniu granicy planszy.

**Zadanie 3. Kontrola złożoności świata.** Rozbuduj interfejs gry o licznik wyświetlający na bieżąco liczbę aktywnych pasów oraz samochodów znajdujących się na planszy.

Wartości powinny aktualizować się automatycznie podczas generowania i usuwania fragmentów świata, a także po ponownym uruchomieniu rozgrywki. Sprawdź, czy liczba aktywnych obiektów nie rośnie w sposób nieograniczony wraz z postępem gracza.

**Wymagania dotyczące sprawozdania**

W sprawozdaniu zbiorczym z **trzech laboratoriów Three.js (Lab 4–6)** należy wyraźnie oznaczyć numer laboratorium oraz numery zadań. Dla każdego zadania umieść krótki opis wprowadzonych zmian, odpowiedni fragment kodu z wyjaśnieniem oraz zrzut ekranu przedstawiający uzyskany efekt. Dla zadania dotyczącego kontroli złożoności dołącz porównanie liczby aktywnych obiektów na początku oraz po dłuższej rozgrywce. Ostateczne sprawozdanie zostanie przesłane po zakończeniu Lab 6 wszystkich trzech spotkań.